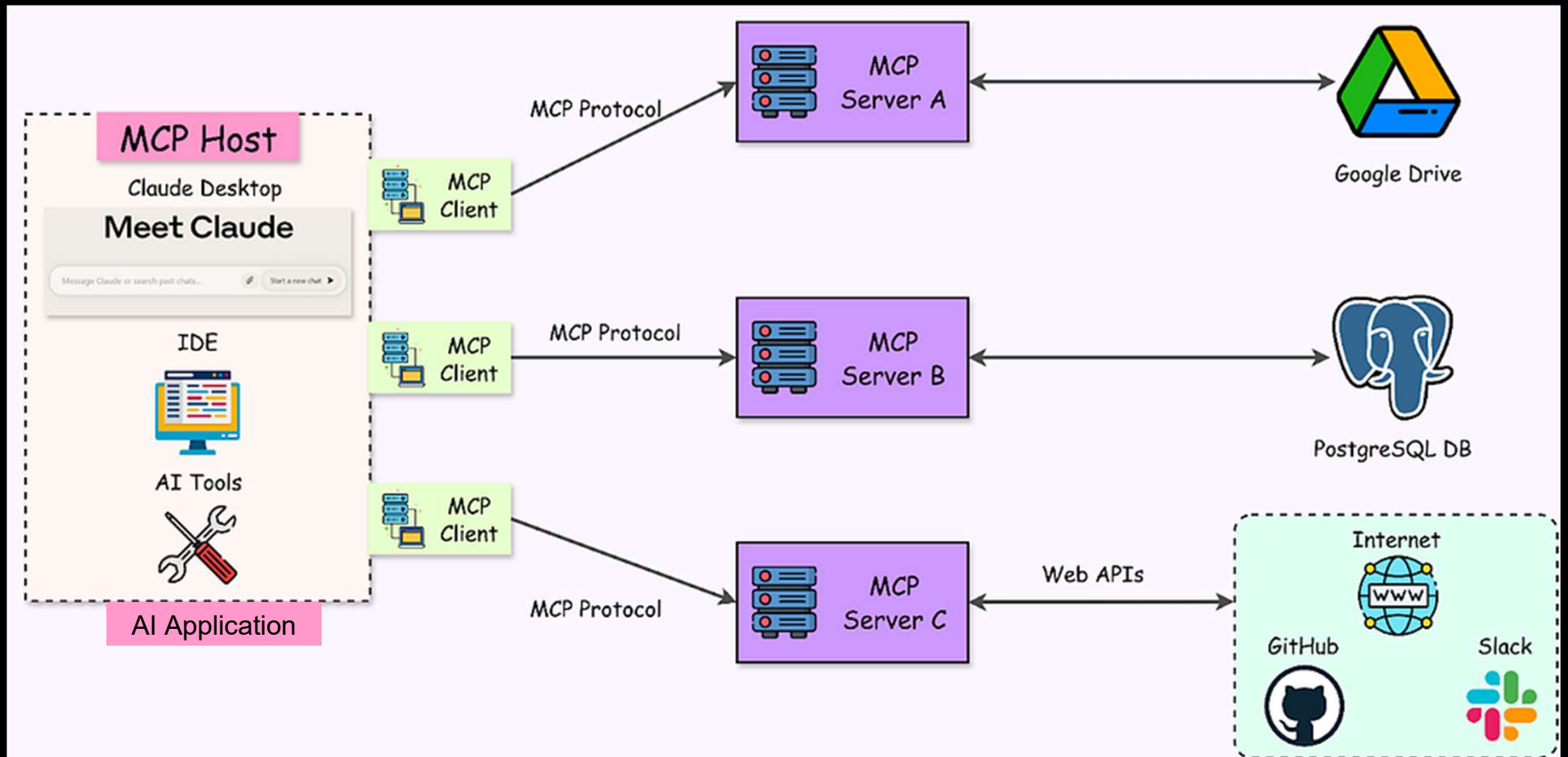


MCP: Model Context Protocol

Models are only as good
as the **context** provided to them.
— Anthropic

MCP Architecture



Before MCP - M x N Problem

The complexity of point-to-point connections



The M x N Problem

⚠ 4 applications × 2 services = **8 connections**

⌘ Each connection requires separate SDKs, authentication, error handling

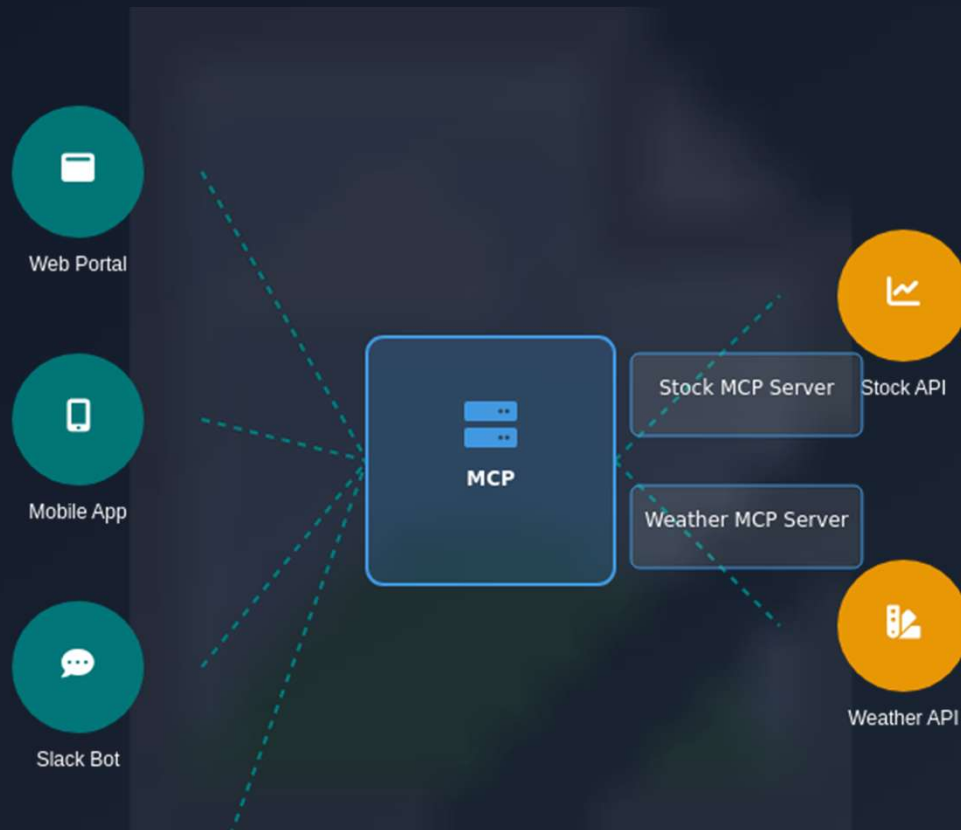
⚡ Different authentication flows: API keys, OAuth, etc.

🔧 Custom response mapping for each service

Each new application or service **exponentially increases** integration

The MCP Solution: Standardized Integration

Transforming $M \times N$ to $M+N$ with standardized clients and servers



6 connections

The M+N Solution

- ✓ 4 applications + 2 services = **6 connections**
- 🔌 Each app uses a standard MCP client interface
- 📦 Each service is wrapped in a standardized MCP server
- 🔄 Uniform protocol for all connections

*New applications or services only require **one new connection** to the MCP ecosystem*

How does MCP solve it?

- MCP Server
 - Register tools using standard metadata
- Client
(such as ChatGPT, any AI Chatbot, vs code or your own AI Application)
 - Discovers the tools given an MCP endpoint
 - Registers the tools with the LLM so that LLM can reason and use them as needed
 - The LLM responds with a tool-call response whenever it needs to invoke a tool
 - The Application/Agent will communicate with the mcp server based on guidance from the LLM

The **Model Context Protocol** standardizes how **AI applications** and **agents** interact with external **tools** and **data sources**.

MCP – Standardized the interface

MCP acts like a “USB port”, providing AI applications with a **universal interface** to access **relevant context** and **perform actions** on other systems.

OpenAI meets MCP

```
from openai import OpenAI

client = openai.OpenAI(
    api_key = 'openai-api-key'
)
```

```
response = client.responses.create(
    model = "o3",
    tools = [
        {
            "type": "mcp",
            "server_label": "mindsdb-mcp",
            "server_url": "https://5a52-88-203-84-191.ngrok-free.app/sse",
            "headers": {"Authorization": "Bearer $MINDSDB_MCP_ACCESS_TOKEN"},
            "require_approval": "never",
        }
    ],
    input = "What tools do you have available?"
)
```

```
>>> print(response)
"""
```

Tool: mcp_mdb.query

- Description: Execute a SQL query against MindsDB.

- Args:

- query: The SQL query to execute

- context (optional): Optional context parameters for the query

- Returns: Dict containing the query results or error information



x



**Specify MindsDB MCP
server as tool**

MCP – server side simplicity

Any team or vendor can spin up an MCP server that exposes one or more capabilities

- GitHub-connector
- Postgres-connector
- shell-executor, etc.

The servers speak the same JSON-RPC wire protocol, so any MCP-aware client can plug in.

MCP – client side simplicity

From the client's perspective

- Just point your MCP SDK at the server's URL (and credentials) and immediately gain all its functions.
- No per-tool SDKs
- No custom wiring
- Enables discovery

Real World scenario – Finance portfolio Q&A App

Scenario:

Your portfolio-management team in a private-bank wants an on-demand assistant that can answer natural-language questions about client portfolios, flag any trades that may breach the bank's risk limits, and generate quick compliance memos.

How MCP helps?

- Data Connectors
 - Postgres/snowflake/sharepoint/S3
- Uniform client SDK
 - Agents use the MCP client APIs to discover and access the services in a uniform manner

Is MCP Server offering same functionality as API Gateways?

Traditional API Gateway / Backend	MCP Server Layer
✓ Full control over routing, auth, rate-limits, SLAs	✓ Leverages that control: proxies through your gateway, reusing the same OAuth/JWT flows
✓ Battle-tested middleware (logging, metrics, retries)	✓ Inherits all of the above—MCP sits on top of your existing services
✗ Requires custom client libraries or code-gen per service	✓ Provides a one-size-fits-all JSON-RPC client schema consumable by any MCP-aware agent
✗ No native “tool discovery” for LLMs	✓ Agents can introspect available methods, parameters and docs at runtime

Using MCP client -

Get your Zapier MCP URL

Sign up or log in at <https://zapier.com/mcp> and generate your **unique** MCP endpoint (it'll look like

In the Zapier UI, scope which actions (tools) you want to expose—e.g.
“send_email”, “create_event”, “search_email”

Install the MCP Python SDK

```
pip install mcp
```

Access the service by using the MCP Client APIs, via the MCP Server

④ List available tools

```
tool_list = (await session.list_tools()).tools
```

Agentic AI – n8n

What is n8n Agentic AI

- n8n is an open-source, "fair-code" workflow automation platform that combines traditional workflow automation with advanced AI capabilities, specifically focusing on agentic AI.
- It allows users to visually design complex AI-powered workflows using a node-based interface, blending AI models, pre-defined logic, custom code, and human-in-the-loop interventions.

N8n – the company

- n8n is a company founded in 2019 by **Jan Oberhauser**.
- It operates with a "fair-code" distribution model, meaning its source code is visible and can be self-hosted, allowing for custom functions, logic, and apps.
- The company has secured significant funding rounds from investors including Sequoia, Felicis Ventures, and Highland Europe.

<https://blog.n8n.io/ai-agentic-workflows/>